AES-128 and SM4 — Algorithm Analysis

A didactic, data-driven overview

PFE 5A — generated from the project's own `src/aes.py` and `src/sm4.py`

This document is produced by `make_analysis.py`: it runs the project's Python implementations, measures how each cipher behaves, and draws the results. Every number and every S-box value below comes from the actual code, not from a textbook table. It covers the *ciphers* only — their structure and their diffusion properties — and contains no fault-injection or key-recovery material.

1 AES-128 at a glance

AES-128 encrypts a 128-bit block in ten rounds. The block is held as a **4×4 matrix of bytes** (the *state*), filled column by column: byte index i sits at row i mod 4, column $i \div 4$. Each round mixes the state with four operations; the final round omits MixColumns.

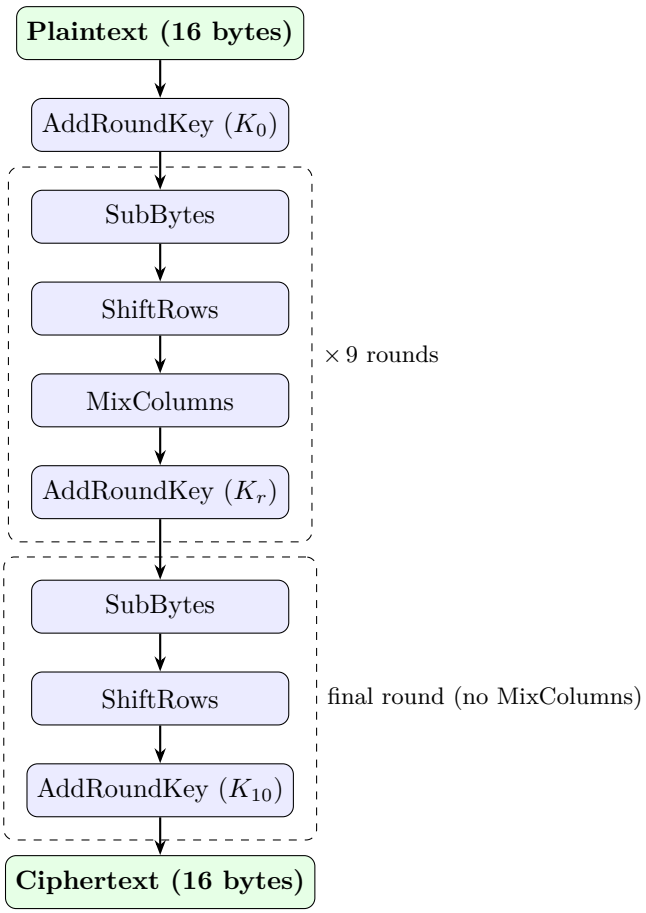


Figure 1: The AES-128 encryption flow. Nine full rounds, then a final round.

Known-answer check. With key `000102...0E0F` and plaintext `001122...EEFF`, the code produces ciphertext `69C4E0D86A7B0430D8CDB78070B4C55A`, matching the FIPS 197 example.

2 The AES S-box

SubBytes replaces every byte through one fixed 16×16 lookup table — the only non-linear step in AES. The heatmap shades each entry by its value (light = low, dark = high); rows are the high nibble, columns the low nibble.

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
1	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
10	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
11	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
12	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
13	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
14	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
15	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

Figure 2: AES S-box. Bijective (a permutation of all 256 bytes). Fixed points: none.

A good substitution box is a bijection with no fixed point ($S(x) = x$) and no opposite fixed point; the AES S-box satisfies this, which is one reason a one-byte change never "stays put".

3 Diffusion — how one bit spreads

Flip a single bit of the plaintext and compare, after every round, against the unmodified encryption. Two measurements: how many of the 16 **bytes** differ, and the Hamming distance in **bits** (out of 128). Full diffusion means about half the bits (64) differ — an attacker learns nothing from the output about which bit changed.

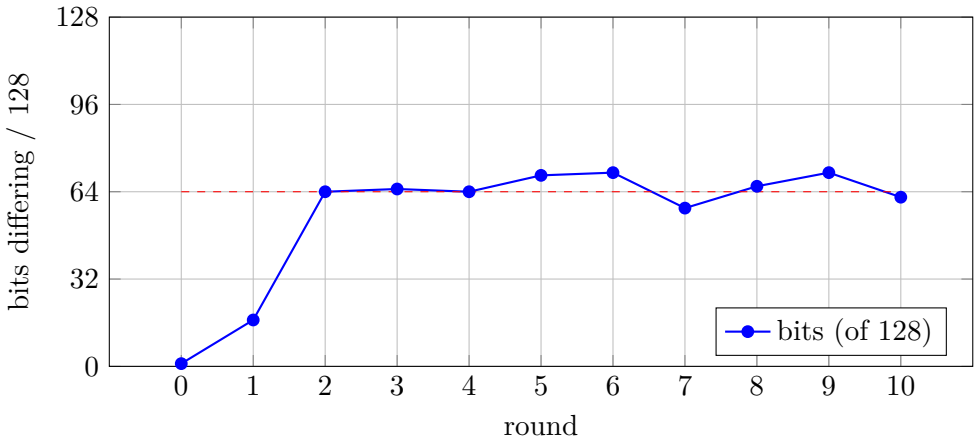


Figure 3: AES avalanche effect. One flipped input bit reaches 17 bits after round 1 and about half (64) by round 2.

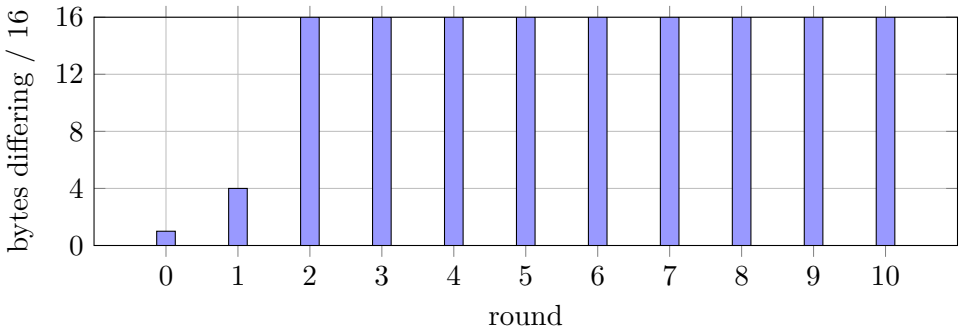


Figure 4: Byte diffusion: MixColumns spreads one changed byte to a full column (4 bytes) in round 1, and ShiftRows + MixColumns reach all 16 bytes by round 2.

4 SM4 at a glance

SM4 is the Chinese national block cipher (GB/T 32907). It also works on 128-bit blocks, but is built as a **32-round Feistel-like network** on four 32-bit words X_0, X_1, X_2, X_3. Each round replaces the oldest word:

$$X_{i+4} \;=\; X_i \oplus T(X_{i+1} \oplus X_{i+2} \oplus X_{i+3} \oplus rk_i)\,,$$

where $T = L(\tau(\cdot))$: τ applies the S-box to each of the four bytes, and L mixes them with rotations $L(b) = b \oplus (b \lll 2) \oplus (b \lll 10) \oplus (b \lll 18) \oplus (b \lll 24)$.

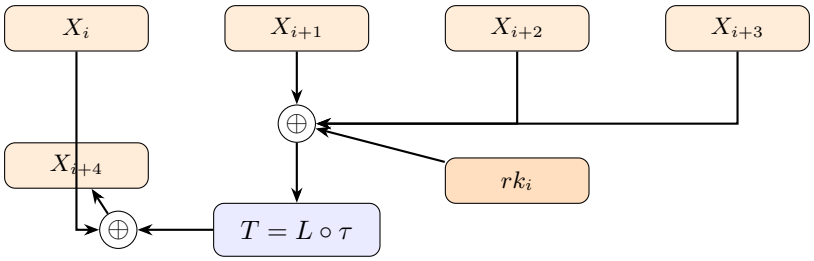


Figure 5: One SM4 round. After 32 rounds the ciphertext is $(X_{35}, X_{34}, X_{33}, X_{32})$.

Known-answer check. With key and plaintext both `0123...3210`, the code produces `536E424686B3E94FD2069`

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	D6	90	E9	FE	CC	E1	3D	B7	16	B6	14	C2	28	FB	2C	05
1	2B	67	9A	76	2A	BE	04	C3	AA	44	13	26	49	86	06	99
2	9C	42	50	F4	91	EF	98	7A	33	54	0B	43	ED	CF	AC	62
3	E4	B3	1C	A9	C9	08	E8	95	80	DF	94	FA	75	8F	3F	A6
4	47	07	A7	FC	F3	73	17	BA	83	59	3C	19	E6	85	4F	A8
5	68	6B	81	B2	71	64	DA	8B	F8	EB	0F	4B	70	56	9D	35
6	1E	24	0E	5E	63	58	D1	A2	25	22	7C	3B	01	21	78	87
7	D4	00	46	57	9F	D3	27	52	4C	36	02	E7	A0	C4	C8	9E
8	EA	BF	8A	D2	40	C7	38	B5	A3	F7	F2	CE	F9	61	15	A1
9	E0	AE	5D	A4	9B	34	1A	55	AD	93	32	30	F5	8C	B1	E3
10	1D	F6	E2	2E	82	66	CA	60	C0	29	23	AB	0D	53	4E	6F
11	D5	DB	37	45	DE	FD	8E	2F	03	FF	6A	72	6D	6C	5B	51
12	8D	1B	AF	92	BB	DD	BC	7F	11	D9	5C	41	1F	10	5A	D8
13	0A	C1	31	88	A5	CD	7B	BD	2D	74	D0	12	B8	E5	B4	B0
14	89	69	97	4A	0C	96	77	7E	65	B9	F1	09	C5	6E	C6	84
15	18	F0	7D	EC	3A	DC	4D	20	79	EE	5F	3E	D7	CB	39	48

Figure 6: SM4 S-box, shaded by value. Bijective. Fixed points: 0xAB (171).

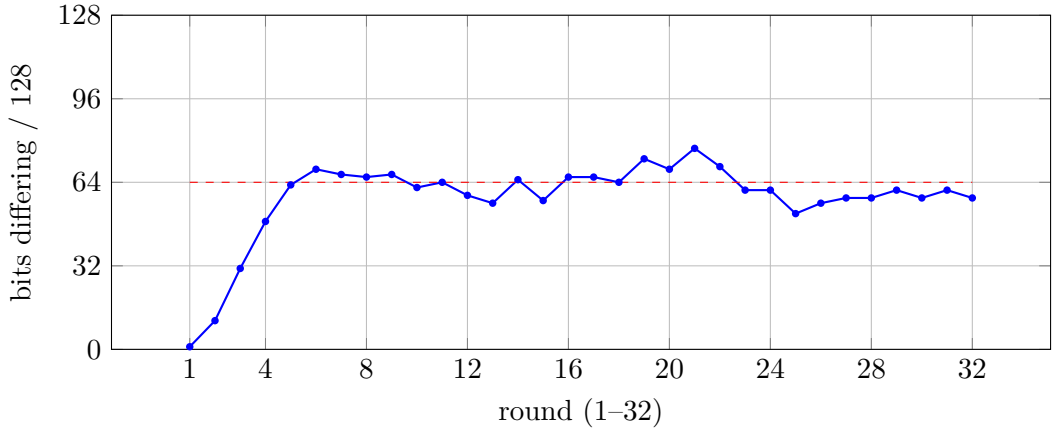


Figure 7: SM4 avalanche effect across its 32 rounds. Diffusion builds more gradually than AES because each round changes only one 32-bit word.

5 AES vs. SM4 — summary

	AES-128	**SM4**
Block / key size	128 / 128 bits	128 / 128 bits
Structure	Substitution–permutation	Feistel-like network
Rounds	10	32
State	4×4 bytes	four 32-bit words
Non-linear step	S-box (SubBytes)	S-box (τ)
Linear step	ShiftRows + MixColumns	rotations L
S-box fixed points	none	0xAB (171)

Table 1: Side-by-side structure, with S-box fixed points measured from the code.

Reproducibility: rerun `python make_analysis.py` to regenerate `analysis/data.tex` and rebuild this PDF. All figures are redrawn from the current `src/` implementations.